

1. Η μεγάλη εικόνα

Το StudyCode είναι Django εφαρμογή. Η βασική ροή είναι:

```
Χρήστης
↓
Browser
↓
URL
↓
urls.py
↓
View
↓
Models / Database
↓
View
↓
Template
↓
HTML
↓
Browser
```

Αυτό είναι ίσως το σημαντικότερο διάγραμμα ολόκληρου του project.

Παράδειγμα: ο μαθητής επισκέπτεται:

```
/courses/python/
```

Το urls.py αποφασίζει:

```
«Αυτό το URL ανήκει στο course_detail view.»
```

Το view παίρνει το course από τη βάση:

```
course = get_object_or_404(
    Course,
    slug=slug,
    published=True,
)
```

και το στέλνει στο template:

```
return render(
    request,
    "courses/course_detail.html",
    {"course": course},
)
```

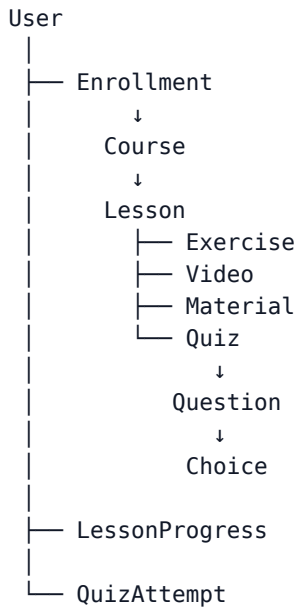
Το template τελικά εμφανίζει:

```
<h1>{{ course.title }}</h1>
```

2. Models — τι αποθηκεύουμε

Τα `models.py` περιγράφουν τη δομή των δεδομένων μας.

Έχουμε, για παράδειγμα:



Για παράδειγμα:

```
class Lesson(models.Model):
    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
        related_name="lessons",
    )
```

σημαίνει:

Ένα `Course` μπορεί να έχει πολλά `Lesson`, αλλά κάθε `Lesson` ανήκει σε ένα `Course`.

Άρα μπορούμε να γράψουμε:

```
course.lessons.all()
```

3. URLs — πού θέλει να πάει ο χρήστης

Το `urls.py` είναι ουσιαστικά ο χάρτης της εφαρμογής.

Παράδειγμα:

```
path(
    "courses/<slug:slug>/",
    views.course_detail,
    name="course_detail",
)
```

Όταν κάποιος επισκεφτεί:

```
/courses/python/
```

το Django παίρνει:

```
slug = "python"
```

και καλεί:

```
course_detail(request, slug)
```

Γι' αυτό στα templates μπορούμε να γράφουμε:

```
{% url 'course_detail' course.slug %}
```

αντί να γράφουμε URLs με το χέρι.

4. Views — η λογική

Τα views είναι ουσιαστικά ο εγκέφαλος ανάμεσα στο URL, τη βάση και το template.

Για παράδειγμα:

```
def course_detail(request, slug):  
  
    course = get_object_or_404(  
        Course,  
        slug=slug,  
        published=True,  
    )  
  
    return render(  
        request,  
        "courses/course_detail.html",  
        {  
            "course": course,  
        },  
    )
```

Έχουμε:

```
request  
↓  
course_detail()  
↓  
Database  
↓  
Course object  
↓  
context  
↓  
course_detail.html
```

5. Templates — τι βλέπει ο χρήστης

Τα templates είναι HTML + Django Template Language.

Για παράδειγμα:

```
<h1>{{ course.title }}</h1>

<p>
    {{ course.description }}
</p>
```

και μπορούμε να έχουμε λογική:

```
{% if user.is_authenticated %}

    Καλώς ήρθες {{ user.username }}

{% else %}

    <a href="{% url 'login' %}">
        Σύνδεση
    </a>

{% endif %}
```

Εδώ όμως δεν βάζουμε την κύρια business logic. Αυτή ανήκει κυρίως στα views/models.

6. base.html

Το base.html είναι ο κοινός σκελετός όλων των σελίδων:

```
base.html
|
├── <head>
├── Bootstrap
├── CSS
├── StudyCode navbar
|
├── {% block content %}
|
├── footer
└── JavaScript
```

Ένα άλλο template κάνει:

```
{% extends "base.html" %}
```

και μετά:

```
{% block content %}

<h1>Τα μαθήματα</h1>

{% endblock %}
```

Έτσι δεν επαναλαμβάνουμε navbar/footer σε κάθε HTML αρχείο.

7. Authentication

Χρησιμοποιούμε το authentication system του Django για:

```
Register  
Login  
Logout  
Password reset  
Authenticated users
```

Όταν κάποιος κάνει login, μπορούμε να χρησιμοποιήσουμε:

```
request.user
```

ή στο template:

```
{{ user.username }}
```

και:

```
{% if user.is_authenticated %}
```

8. Enrollment

Το Enrollment συνδέει:

User ↔ Course

Έχουμε:

```
user = models.ForeignKey(...)
course = models.ForeignKey(...)
```

Άρα μπορούμε να ρωτήσουμε:

```
Enrollment.objects.filter(
    user=request.user,
    course=course,
).exists()
```

Με απλά λόγια:

«Ο συγκεκριμένος μαθητής είναι εγγεγραμμένος σε αυτό το course;»

9. Permissions

Εδώ πήγαμε ένα βήμα παραπέρα.

Δεν αρκεί να κρύψουμε ένα κουμπί από το HTML.

Κάποιος θα μπορούσε να γράψει απευθείας το URL.

Γι' αυτό έχουμε:

```
@login_required
```

και επιπλέον:

```
is_enrolled = Enrollment.objects.filter(
    user=request.user,
    course=lesson.course,
).exists()

if not is_enrolled:
    return redirect(...)
```

Άρα:

```
Logged out
```

```
↓
```

```
❑ Lesson
```

```
Logged in αλλά όχι enrolled
```

```
↓
```

```
❑ Lesson
```

```
Logged in + enrolled
```

```
↓
```

```
❑ Lesson
```

Αυτό είναι πραγματικό backend access control.

10. Lesson progress

Με το:

```
LessonProgress
```

αποθηκεύουμε κάτι σαν:

```
Γιάννης  
Python  
Lesson 1  
completed = True
```

Έτσι η πρόοδος δεν υπάρχει μόνο στην οθόνη.

Αποθηκεύεται στη βάση.

Και γι' αυτό μετά μπορούμε στο profile να υπολογίζουμε:

```
Python  
██████████░░░░ 80%
```

11. Quiz

Η δομή μας είναι:

```
Quiz
↓
Question
↓
Choice
```

και κάθε Choice έχει:

```
is_correct = True
```

ή:

```
is_correct = False
```

Όταν γίνεται POST το quiz, παίρνουμε την επιλογή του χρήστη:

```
request.POST.get(
    f"question_{question.id}"
)
```

τη συγκρίνουμε με τη σωστή απάντηση και αυξάνουμε:

```
score += 1
```

12. QuizAttempt

Δεν θέλαμε απλώς να εμφανίζεται:

8/10

και μετά να χάνεται.

Δημιουργήσαμε:

QuizAttempt

ώστε να αποθηκεύουμε:

User

Quiz

Score

Total Questions

Percentage

Date

Άρα έχουμε πλέον ιστορικό επιδόσεων μαθητή.

13. Profile

Το profile συγκεντρώνει πληροφορίες από διαφορετικά models:

```
User
|
├─ Enrollments
├─ LessonProgress
└─ QuizAttempts
```

Εδώ φαίνεται γιατί η σωστή σχεδίαση της database είναι τόσο σημαντική.

Το profile δεν χρειάζεται να αποθηκεύει ξανά όλες αυτές τις πληροφορίες. Τις διαβάζει από τις υπάρχουσες σχέσεις.

14. Messages

Με:

```
messages.success(...)  
messages.error(...)  
messages.warning(...)
```

το backend μπορεί να ενημερώσει τον χρήστη.

Για παράδειγμα:

```
messages.success(  
    request,  
    "Η εγγραφή στο μάθημα ολοκληρώθηκε."  
)
```

και το base.html αναλαμβάνει να το εμφανίσει.

15. Tests

Τα tests λειτουργούν σαν αυτόματος ελεγκτής.

Αντί να κάνουμε χειροκίνητα:

```
login  
→ enroll  
→ lesson  
→ complete  
→ quiz  
→ profile
```

τρέχουμε:

```
python manage.py test
```

και το Django ελέγχει αυτόματα τις βασικές ροές.

Αυτό είναι πολύ σημαντικό όταν αργότερα αλλάζουμε κώδικα.

16. SQLite και PostgreSQL

Τοπικά:

```
Django  
↓  
SQLite  
↓  
db.sqlite3
```

Production:

```
Django / Azure  
↓  
PostgreSQL  
↓  
Production database
```

Το SQLite είναι εξαιρετικό για development.

Το PostgreSQL είναι καταλληλότερο για την production εφαρμογή μας.

17. Environment variables

Δεν θέλουμε:

```
SECRET_KEY = "my-real-secret"
```

μέσα στο GitHub.

Γι' αυτό έχουμε:

```
SECRET_KEY = os.environ.get(  
    "DJANGO_SECRET_KEY"  
)
```

Το ίδιο concept χρησιμοποιούμε για:

```
SECRET_KEY  
DEBUG  
ALLOWED_HOSTS  
DATABASE_URL  
email credentials
```

Άρα:

```
GitHub  
→ περιέχει κώδικα
```

```
Azure Environment  
→ περιέχει secrets
```

18. GitHub

Το GitHub δεν είναι απλώς backup.

Μας δίνει:

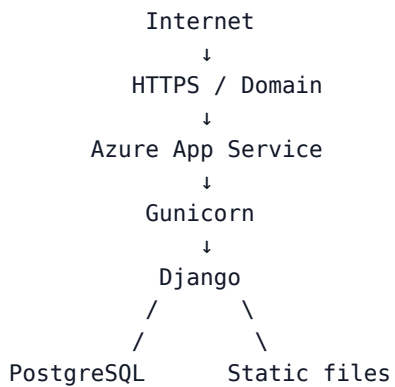
- Version control
- Commit history
- Portfolio
- Deployment source
- Collaboration

και με .gitignore κρατάμε έξω:

- .env
- .venv/
- db.sqlite3
- __pycache__/

19. Azure

Όταν εγκριθεί το quota, η τελική αρχιτεκτονική θα είναι περίπου:



και το GitHub θα χρησιμοποιείται για να μεταφέρουμε τον κώδικά μας προς το production environment.

20. Το σημαντικότερο που θέλω να κρατήσεις

Αν σε μια συνέντευξη σου πουν:

«Περιέγραψέ μου το StudyCode.»

Θέλω να μπορείς να πεις περίπου:

Το StudyCode είναι μια εκπαιδευτική web εφαρμογή που ανέπτυξα με Django. Χρησιμοποιεί Django authentication για διαχείριση χρηστών, relational models για courses, lessons και enrollments, αποθηκεύει την πρόοδο των μαθητών και τις προσπάθειες στα quizzes και εφαρμόζει access control ώστε το εκπαιδευτικό περιεχόμενο να είναι διαθέσιμο στους κατάλληλους χρήστες. Έχω προσθέσει automated tests, environment-based configuration και PostgreSQL configuration για production, ενώ το deployment γίνεται σε Azure App Service.

Αυτό πλέον περιγράφει ολόκληρο web application, όχι απλώς μερικές Django σελίδες.

Και η πιο σημαντική αλυσίδα που πρέπει να θυμάσαι είναι:

```
URL
↓
View
↓
Model / Database
↓
Context
↓
Template
↓
Response
```

Αν κατανοήσεις πραγματικά αυτή την αλυσίδα, έχεις καταλάβει τον πυρήνα του Django.